

关闭



...

漏洞描述

根本原因

```
output = {
    "output": raw_output,
    "returncode": -1,
    "exception_info": f"执行命令时发生错误: {e}",
    ...
}
```

self._check_finished (输出)
返回输出 # 默默吞咽

影响

- 在基准测试中，在step_limit=500的条件下，容器在第200步失效的任务将继续执行300步，消耗约300个不必要的API调用及相关成本。
- 轨迹从未被正确保存，且没有合适的退出状态。

拟议修复方案

1. 在 exceptions.py 中添加 ContainerNotRunning 例外子类 InterruptAgentFlow:

```
class ContainerNotRunning (InterruptAgentFlow):  
    """当 Docker 容器不再运行时，会被提升。”
```

2. 在 execute ()， 中检测正常返回路径和异常路径中的容器死标记:

```
_CONTAINER_DEAD_MARKERS = (“无此类容器”、“未运行”、“无此类容器”)
```

After Subprocess.run 返回

```
如果result.returncode != 0 且 any (m in result.stdout for m in m in _CONTAINER_DEAD_MARKERS):  
    raise ContainerNotRunning (  
        {"role": "exit", "content": result.stdout,  
        "extra": {"exit_status": "ContainerNotRunning", "submission": ""}}  
    )
```

除了块外

```
如果有 (m in str (e) for m in _CONTAINER_DEAD_MARKERS):  
    raise ContainerNotRunning (  
        {"role": "exit", "content": str (e),  
        "extra": {"exit_status": "ContainerNotRunning", "submission": ""}}  
    ) 由于
```

ContainerNotRunning 是 InterruptAgentFlow 子类，agent 的 run () 循环捕获该程序，记录退出消息，finally: self.save () 块保存轨迹——这与 LimitsExceed 使用的干净关机路径完全相同。

修复后的结果

- 当容器失效时，代理会立即终止
- 轨迹保存后exit_status: "ContainerNotRunning"
- 算作失败实例（未提交），没有浪费的API调用

  ZZ0803 新增内容 漏洞 on Apr 2

  GopalGB 于on 4月4日日添加了一个提交，涉及此问题